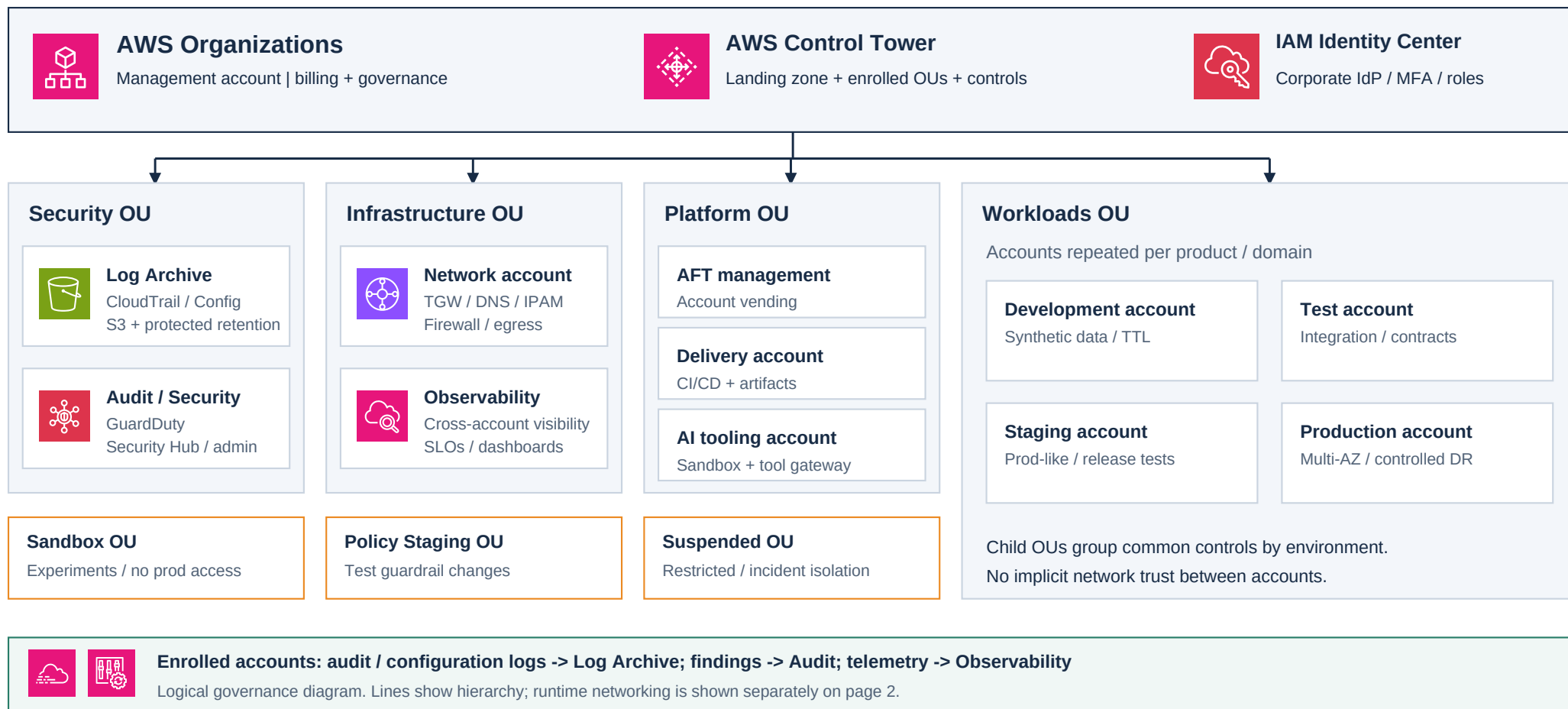


# Governed landing zone, autonomous product teams

Shared governance and account foundations; applications run in isolated product and environment accounts. [1-4]

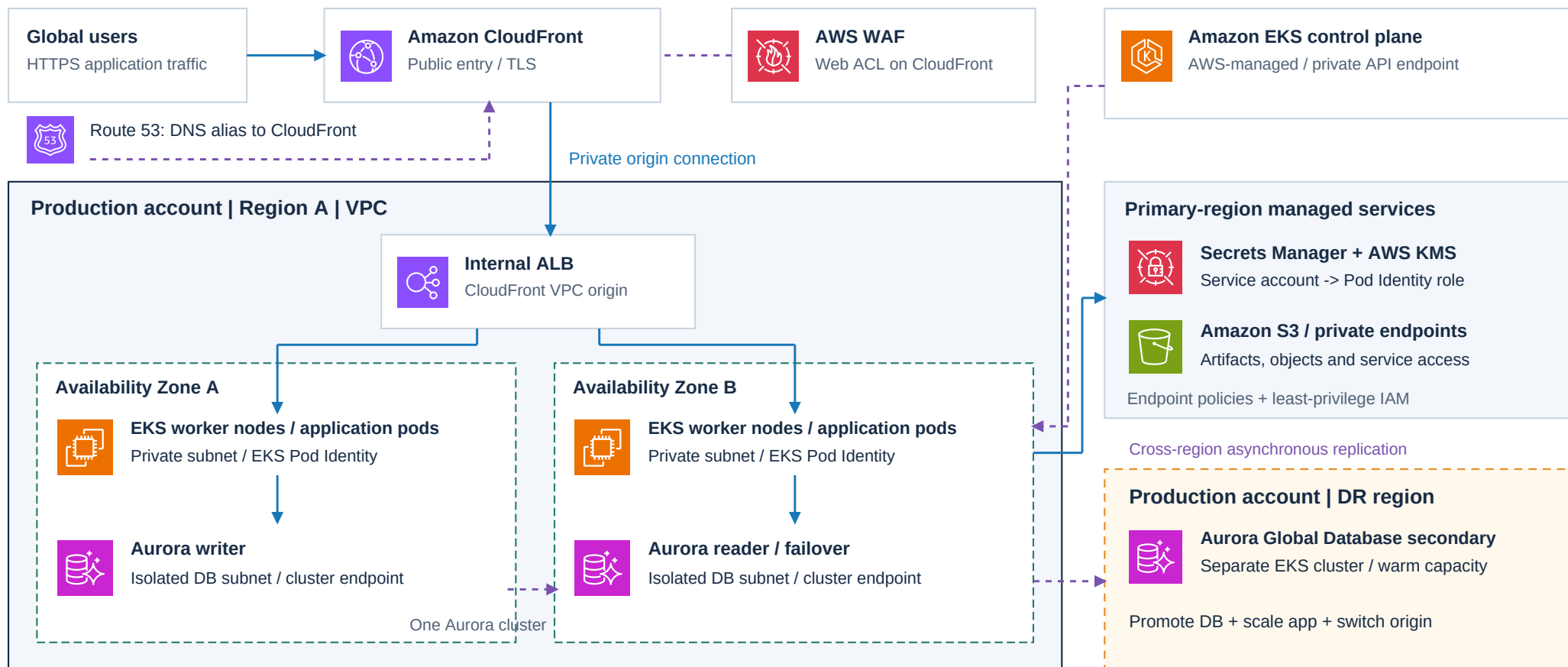


**Design decision.** AWS Control Tower establishes a landing zone on AWS Organizations. The management account is reserved for governance. The Security OU separates audit storage from security tooling. OUs group common controls; accounts provide isolation, independent quotas, and cost attribution.

Use separate Development, Test, Staging, and Production accounts per product or domain. Size account boundaries by ownership and risk, not engineer count. Each environment has its own EKS cluster and data. Platform automation provisions and enrolls additional OUs/accounts; Control Tower does not create the application platform automatically.

# Amazon EKS: private workloads, multi-AZ resilience

Managed Kubernetes control plane with EC2 workers in private subnets. Two worker AZs are shown; use additional AZs where required. [5,8-10]



Blue: application traffic    Purple dashed: management / DNS / replication    One regional EKS cluster spans the worker AZs.

**Request path.** Route 53 resolves the CloudFront domain. AWS WAF protects CloudFront; a private VPC origin connects to the internal ALB. AWS Load Balancer Controller manages Ingress and registers pod IP targets. Pods use Aurora writer/reader endpoints as appropriate and EKS Pod Identity for AWS service access.

**Control and recovery.** The EKS API is private and reached through approved network paths. The control plane manages workers; it does not carry application traffic. Multi-AZ protects against zonal failure. Regional recovery requires a separate EKS cluster, database promotion, and a controlled origin switch; it is not automatic write failover.

# Architecture decisions and operating boundaries

Security, networking, Kubernetes operations, reliability, and cost decisions. Numerical recovery targets remain assessment assumptions.

## 01 Account vending and governance

The developer portal opens an account-request PR with owner, cost center, data classification, and environment. AFT provisions and customizes accounts from its dedicated account. Separate pipelines deploy EKS infrastructure and applications. Versioned templates configure baseline IAM, logging, network controls, and metadata. Test policy changes in the Policy Staging OU before rollout. [1,3]

## 02 Identity and security boundaries

Federate IAM Identity Center to the corporate IdP with MFA. Use EKS access entries and scoped Kubernetes RBAC; production elevation requires time-limited approval. SCPs limit maximum permissions, grant no access, and exclude the management account. Run Terraform policy checks separately from CloudFormation-based proactive controls. Use Pod Identity per service account; restrict node-role/IMDS access. Enforce Pod Security Admission and default-deny NetworkPolicy with a supporting CNI. [4,10,11]

## 03 Private networking and ingress

The Network account owns IPAM, DNS Resolver, TGW, and inspected egress. Separate production/nonproduction routing; no implicit cross-environment access. Use SGs between tiers, endpoint policies, and VPC CNI IP-capacity planning. Private EKS endpoints require reachable DNS/network paths for runners and operators. CloudFront VPC origins require supported regions/protocols and an attached internet gateway, but no public route to the origin subnet. [5,9]

## 04 EKS capacity and lifecycle

Use an On-Demand managed node group for system controllers and Karpenter NodePools for application capacity. HPA scales pods; Karpenter scales its EC2 nodes. Define resource requests/limits, topology spread, readiness/startup probes, and PodDisruptionBudgets. PDBs limit voluntary disruption, not AZ failures. Test EKS, add-on, node-image, and controller upgrades in Staging before rolling or blue/green production upgrades. [8]

## 05 Availability and regional recovery

Spread replicas across worker AZs and retain spare capacity. Use Aurora writer/reader instances across AZs. Example critical-tier targets: SLO 99.9%, RTO 60 minutes, RPO 15 minutes. Prepare a separate regional EKS cluster, replicated ECR images, manifests, secrets, and Aurora Global Database secondary. Test database promotion, traffic switching, and restores; use cross-account backups. Validate region/engine support and replication lag. [6]

## 06 Ownership and cost management

Platform teams own cluster lifecycle and approved add-ons; product teams own workloads, SLOs, on-call, and cost. Each environment has separate IAM roles, secrets, data, and IaC state. Namespaces organize trusted workloads; they do not replace account/cluster isolation. Track EKS control-plane, EC2, NAT, endpoint, logging, and DR costs. Use quotas, rightsizing, TTL, and interruption-tolerant Spot capacity; budget alerts are not spending caps. [7]

# Environment strategy and governed delivery

Consistent platform controls with progressively stronger access, capacity, and release assurance toward production.

| Environment | Data and access                                     | Deployment and reliability                         |
|-------------|-----------------------------------------------------|----------------------------------------------------|
| Development | Synthetic data; scoped developer access             | Automated builds; namespace TTL; small capacity    |
| Test        | Synthetic/masked data; separate identities          | Integration, contract, and negative security tests |
| Staging     | Masked data; production-like EKS configuration      | Load, DAST, upgrade, rollback, and multi-AZ tests  |
| Production  | Business data; least privilege; temporary elevation | Protected release gates; multi-AZ; tier-based DR   |

## Kubernetes delivery and observability

Build once; run tests, SAST/SCA, secret/IaC/image scans, and generate an SBOM. Sign images in ECR and promote the same digest with versioned Helm/Kustomize manifests. A scoped GitOps controller per cluster reconciles approved releases. Gate production on artifact/plan/target approval and SLO checks; roll back manifests/images separately from database migrations.

Collect pod/node metrics, application logs, and OpenTelemetry traces. Enable EKS control-plane/audit logs; CloudTrail does not capture every Kubernetes API action. Central dashboards show SLOs, owner, deployments, and cost. Protect audit retention, redact sensitive data, and route actionable alerts to the service owner.

## AI agents: controlled execution

Run agents in the isolated AI tooling account with short-lived role sessions, a sandbox, and an external policy-enforcing tool gateway. Agents may create code/PRs but cannot approve their own changes, grant IAM/RBAC privileges, or bypass admission policies. Production and destructive actions require approval bound to the exact action and target.

Record requester, agent/task ID, approval, tool call, Kubernetes/cloud target, outcome, and cost. Emergency controls block the gateway, stop jobs, isolate networking, and revoke supported sessions. Removing a trust policy alone does not invalidate all active sessions. Apply cost/time limits, egress allowlists, and sensitive-data redaction.

## Official references and design assumptions

- [\[1\] Control Tower: account structure](#)  
[\[2\] AWS Architecture Icons](#)  
[\[3\] AFT overview and scope](#)  
[\[4\] Control Tower control behavior](#)
- [\[5\] CloudFront private VPC origins](#)  
[\[6\] Aurora Global Database](#)  
[\[7\] AWS Well-Architected Framework](#)  
[\[8\] EKS application reliability / scaling](#)
- [\[9\] EKS load balancing](#)  
[\[10\] EKS Pod Identity](#)  
[\[11\] EKS security best practices](#)

Assumptions: containerized web/API workloads; regions, residency, traffic, and business SLAs are not yet agreed. This high-level design requires detailed routing, backup, EKS sizing, and deployment validation. Official icons do not imply AWS certification or endorsement.